

# Simulador de Sistemas de Archivos: FAT vs Ext2

Brigido Noguera  
Angel Besteiro

9 de octubre, 2025

## 1 Aspectos generales

## 2 FAT

- Simulación
- Ventajas
- Desventajas
- Usos Realistas

## 3 Ext2

- Simulación
- Ventajas
- Desventajas
- Usos Realistas

## 4 Conclusión





# Simulación FAT: Estructuras Clave

## Definición en proyecto.h

La simulación usa un `std::vector<int>` para la tabla FAT y un `std::map` para el directorio.

```
1 class FATFileSystem : public FileSystem {
2 private:
3     // La tabla FAT: cada indice corresponde a un bloque.
4     std::vector<int> fat_table;
5
6     // Directorio: mapea nombres de archivo a su bloque de
7     inicio.
8     std::map<std::string, int> directory;
9 public:
10    FATFileSystem(std::shared_ptr<Disk> d);
11    // ...
12};
```

## Constantes de Control

# Simulación FAT: Lógica de Creación

## Pasos de create en proyecto.cpp

- 1 Calcular y encontrar bloques libres.
- 2 Enlazar los bloques en la `fat_table`, creando la lista enlazada.
- 3 El último bloque apunta a `FAT_EOF`.
- 4 Escribir el contenido en los bloques del disco.
- 5 Registrar el archivo en el directorio con su bloque de inicio.

## Fragmento de código: Enlazando bloques

```
1 // Enlaza los bloques en la tabla FAT.
2 for (size_t i = 0; i < free_blocks.size() - 1; ++i) {
3     fat_table[free_blocks[i]] = free_blocks[i + 1];
4 }
5 fat_table[free_blocks.back()] = FAT_EOF;
6
7 // Registra el archivo en el directorio.
8 directory[filename] = free_blocks[0];
9
```

# Simulación FAT: Lógica de Recorrido (Lectura)

## Pasos de read en proyecto.cpp

El recorrido es puramente secuencial. Se parte del bloque inicial y se sigue la cadena en la fat\_table hasta encontrar FAT\_EOF.

## Fragmento de código: Lectura secuencial

```
1 std::string FATFileSystem::read(const std::string& filename)
  {
2     if (!directory.count(filename)) {
3         return "Error: El archivo no existe.";
4     }
5
6     std::string content = "";
7     int current_block = directory[filename];
8
9     // Sigue la cadena en la tabla FAT hasta llegar al final
10    .
11    while (current_block != FAT_EOF) {
12        content += disk->readBlock(current_block);
13        current_block = fat_table[current_block];
14    }
```

# Ventajas de FAT

- **Alta Compatibilidad:** Universalmente reconocido por Windows, macOS y Linux de forma nativa.
- **Baja Sobrecarga:** Su estructura simple consume muy poco espacio de metadatos, ideal para dispositivos con bajos recursos.
- **Simplicidad:** Fácil de implementar y recuperar en caso de errores. El mantenimiento del código es más sencillo.
- **Velocidad en Archivos Pequeños:** La lectura es rápida si los archivos ocupan pocos bloques contiguos.



# Desventajas de FAT

- **Bajo Rendimiento:** La lectura es secuencial  $O(N)$ . Se debe recorrer bloque a bloque, lo que es muy lento en archivos grandes y fragmentados.
- **Riesgo de Corrupción:** No tiene registro de transacciones (journaling). Un fallo de energía puede corromper la tabla FAT fácilmente.
- **Límites de Tamaño:** Versiones como FAT32 limitan los archivos a 4GB, insuficiente para aplicaciones modernas (video, bases de datos).
- **Espacio Inútil:** La fragmentación interna y el tamaño fijo de los clústeres pueden desperdiciar mucho espacio.

- **Unidades Flash y Dispositivos de Memoria:** Es el estándar de facto para pendrives, tarjetas SD, cámaras y GPS debido a su compatibilidad universal.
- **Sistemas Embebidos:** Común en firmware de routers, impresoras y microcontroladores por su simplicidad y bajo consumo de recursos.
- **Particiones de Arranque (EFI):** La partición de arranque en la mayoría de los PCs modernos está formateada en FAT32 porque el firmware UEFI la reconoce nativamente.



# Simulación Ext2: Estructura del Inodo

## Definición del Inodo en proyecto.h

El inodo contiene metadatos y los punteros que son la clave de su funcionamiento.

```
1 struct Inode {  
2     bool is_used = false;  
3     int size = 0;  
4     // Punteros directos a bloques de datos.  
5     std::vector<int> direct_pointers;  
6     // Puntero a un bloque que contiene mas punteros.  
7     int single_indirect = -1;  
8  
9     Inode() : direct_pointers(NUM_DIRECT_POINTERS, -1) {}  
10 };  
11
```

# Simulación Ext2: La Tabla de Inodos

## Declaración de Estructuras Clave en proyecto.h

Las estructuras centrales de Ext2 son la Tabla de Inodos y el Directorio.

```
1 // --- Implementación del Sistema de Archivos Ext2 ---
2 class Ext2FileSystem : public FileSystem {
3 private:
4     // Tabla de inodos: un vector donde cada inodo describe
    un archivo.
5     std::vector<Inode> inode_table;
6     // Directorio: mapea nombres de archivo a su número de
    inodo.
7     std::map<std::string, int> directory;
8     // ...
9 };
10
```

## Propósito

- **inode\_table:** Es el arreglo principal de metadatos. Cada entrada (*Inodo*) almacena los atributos del archivo y sus punteros a bloques

# Simulación Ext2: Lógica de Acceso a Punteros

## Lectura en proyecto.cpp

El acceso es jerárquico: primero se leen los bloques directos y luego, si es necesario, se procesa el bloque de punteros indirectos.

## Fragmento de código: Acceso a datos

```
1 // 1. Leer desde punteros directos.
2 for (int block_ptr : inode.direct_pointers) {
3     if (block_ptr != -1) {
4         content += disk->readBlock(block_ptr);
5     }
6 }
7 // 2. Leer desde puntero indirecto simple.
8 if (inode.single_indirect != -1) {
9     std::string pointer_block_content =
10         disk->readBlock(inode.single_indirect);
11     // ...se procesa el bloque de punteros...
12 }
13 // Recorta el contenido al tamaño real del archivo.
14 return content.substr(0, inode.size);
```

# Ventajas de Ext2

- **Acceso Rápido:** El acceso a cualquier bloque requiere un número pequeño y constante de pasos (desreferencias), en lugar de un recorrido secuencial.
- **Gestión de Archivos Grandes:** El mecanismo de punteros indirectos permite manejar archivos enormes de forma eficiente y escalable.
- **Mejor Integridad de Metadatos:** Separar los metadatos en el inodo permite un acceso y modificación más seguros y eficientes.
- **Menos Fragmentación:** La asignación jerárquica reduce el impacto de la fragmentación, ya que la velocidad de acceso no depende de la distancia física entre bloques.

## Desventajas de Ext2

- **Falta de Journaling:** Al igual que FAT, no tiene un registro de transacciones. Un fallo de sistema requiere una revisión completa del disco (`fsck`), que puede ser muy lenta.
- **Mayor Sobrecarga:** Las estructuras de inodos y punteros consumen más espacio en disco para la gestión de metadatos que FAT.
- **Complejidad:** La lógica para manejar la estructura de punteros es significativamente más compleja, lo que dificulta su implementación y mantenimiento.
- **Tiempo de Eliminación:** Borrar un archivo muy grande puede ser lento, ya que se deben liberar todos los bloques de datos y también los bloques de punteros indirectos.



- **Distribuciones Clásicas de Linux:** Fue el sistema de archivos por defecto en muchas distribuciones de Linux (como Debian y Red Hat) durante los años 90, antes de ser sucedido por Ext3 y Ext4.
- **Particiones Varias:** A veces se sigue usando en particiones que no requieren journaling, como una partición /boot, o en sistemas donde la velocidad de escritura es crítica y el riesgo de corrupción es bajo.
- **Almacenamiento de Red (NAS):** Puede encontrarse en servidores de archivos o dispositivos NAS más antiguos o de bajo rendimiento donde la simplicidad y la velocidad sin journaling son aceptables.

## Rendimiento y Escalabilidad

El proyecto de simulación demuestra que la elección de la estructura de metadatos es el factor más crítico en el diseño de un sistema de archivos, determinando su rendimiento y escalabilidad.

- La **lista enlazada de FAT** es simple pero no escala, resultando en un rendimiento pobre para archivos grandes.
- La **estructura de inodos de Ext2** proporciona un acceso rápido y eficiente, sin importar el tamaño del archivo, a costa de una mayor complejidad.

El simulador ilustra de manera práctica por qué las arquitecturas modernas favorecen los Inodos (o estructuras similares) para asegurar la robustez y el alto rendimiento que exigen los entornos informáticos actuales.